

Introdução ao Python e Web Scraping

Gabriel Sanábio Mazzetti Affonso

2026-09-05

Índice

Prefácio	3
1 Introdução	4
1.1 A importância do Python na Estatística	4
1.2 O escopo deste material	4
2 Configurando o Ambiente de Trabalho	6
2.1 O Ambiente <i>Python</i> e o pacote <i>reticulate</i>	6
2.2 Gerenciamento de Bibliotecas com o <i>pip</i>	6
2.3 Instalação Direta em Blocos de Código	6
3 Fundamentos	8
3.1 Filosofia, Sintaxe e Tipagem da Linguagem	8
3.2 Estruturas de Dados Nativas	9
3.3 Estruturas de Controle de Fluxo	9
3.4 Funções e Ferramentas Funcionais	10
3.5 Aplicação Prática: Manipulação de Listas	10
4 Análise Exploratória e Manipulação de Dados em <i>Python</i>	12
4.1 A Biblioteca <i>Pandas</i> e as Estruturas de Dados	12
4.2 Importação e Tratamento de Dados (<i>Data Wrangling</i>)	13
4.3 Visualização dos Dados	13
4.4 Estudos de Caso Práticos	14
4.4.1 Análise de Dados Categóricos: <i>Titanic</i>	14
4.4.2 Análise de Dados Contínuos: <i>Iris</i>	16
5 Referências	19

Prefácio

O presente material consiste na consolidação dos resultados de pesquisa obtidos durante o período de Iniciação Científica, no contexto do projeto **Métodos de Aprendizado Estatístico para Discriminação e Classificação: Aplicações à Indústria 4.0 – Fase 3**. Este documento visa não só o detalhamento dos métodos de coleta de dados via *Web Scraping*, mas também a viabilidade do uso do *Python* para tal, além de destacar as especificidades da linguagem e suas vantagens para o aprendizado de métodos estatísticos e computacionais.

Neste contexto, vale ressaltar que o material é de cunho introdutório, buscando viabilizar o aprendizado do *Python* enquanto linguagem de programação e do *Web Scraping* como técnica poderosa de coleta de dados. Dessa forma, seu foco é desenvolver, no estatístico, o interesse pelos diversos recursos computacionais que podem auxiliá-los enquanto profissionais e estudiosos da área..

A motivação para o desenvolvimento deste material surgiu da necessidade de diversificar as ferramentas de aplicação computacional existentes para analisar dados. A interseção entre a teoria estatística e a aplicação dos métodos se sustenta no casamento entre o rigor matemático e a modernidade computacional. Qualquer das partes que for negligenciada ocasionará problemas ou limitações importantes.

Ademais, é válido ressaltar a seção extra presente no material, desenvolvida com o intuito de explicar a estrutura dos livros digitais, como esse, usando o *Quarto*. Ainda que muitos considerem adequado o uso do *Bookdown* para este tipo de trabalho (o que é totalmente válido, dadas as especificidades do Rmarkdown), o *Quarto* se mostrou uma espécie de sucessor natural muito adequado, apresentando ferramentas modernas e extremamente úteis para detalhes de *layout*, especialmente tratando de documentos compilados em *HTML*.

Por fim, deixo aqui minha gratidão à Universidade Federal de Juiz de Fora (UFJF) pelo ambiente acadêmico. Agradeço, especialmente, ao meu orientador, Professor Doutor Lupércio França Bessegato, pela oportunidade e pelo acompanhamento técnico que possibilitaram o desenvolvimento deste trabalho.

1 Introdução

Na prática, a estatística contemporânea vai muito além do domínio teórico dos fundamentos, da modelagem probabilística e da inferência. A sociedade atual convive com uma realidade em que os dados são o produto mais valioso existente, além simplesmente de fontes de informação. Dessa forma, observa-se nos últimos anos o surgimento de conceitos como o *Big Data*, para descrever o fenômeno da quantidade massiva de dados que alteram significativamente o fluxo de trabalho do analista de dados, exigindo deste profissional, muitas vezes, muito além da capacidade de analisar dados. É necessário ter aptidão para manusear os dados apropriadamente, buscá-los em diferentes fontes, criar filtros específicos relativos às necessidades de um determinado problema, entre outras habilidades.

Neste contexto, este material foi elaborado com o intuito de introduzir a linguagem *Python* no arsenal analítico de estudantes e profissionais de Estatística, demonstrando sua viabilidade e potência para a análise de dados e a automação de rotinas de trabalho.

1.1 A importância do Python na Estatística

Tradicionalmente, a linguagem *R* ocupa um lugar importante na formação do estatístico, por ser uma linguagem vetorizada e já desenvolvida para o trabalho com dados. Porém, apesar do poder do *R* e seus pacotes nos trabalhos de modelagem de dados e no uso de testes clássicos (paramétricos e não-paramétricos), o *Python* consolidou-se como uma linguagem extremamente versátil, que permite aplicações robustas (como as de *Machine Learning*, por exemplo) e a integração de sistemas.

Em termos de aprendizado, o *Python* é facilmente assimilado por quem já teve contato com algoritmos, por conta de sua similaridade com linguagens compiladas, como no caso do *C++* (exigindo apenas adaptações sintáticas), e com linguagens interpretadas, como o *R*.

Na prática, o *Python* apresenta sua maior vantagem ao analisarmos sua quantidade de usuários e, consequentemente, sua quantidade de bibliotecas desenvolvidas para diferentes aplicações. No contexto da Estatística, destacamos o *pandas*, para estruturação de dados tabulares, e o *numpy*, para computação numérica de alta performance. O domínio dessas ferramentas permite ao analista processar grandes volumes de dados com eficiência, além de explorar diferentes possibilidades, como a de romper com a dependência de bancos de dados pré-estruturados através da automatização da coleta de dados via *internet*.

1.2 O escopo deste material

Este material tem um teor introdutório e prático. Ele não tem a intenção de esgotar as funcionalidades do *Python* (que seria, inclusive, uma tarefa praticamente impossível), tampouco de aprofundar-se nas teorias estatísticas que fundamentam nossas análises ou na teoria computacional

que fundamenta e explica as especificidades da linguagem. O objetivo central é ser uma ponte rápida para profissionais de Estatística compreenderem a linguagem e o ecossistema, e como podem se beneficiar deles.

Neste âmbito, a abordagem adotada se resume em execução de códigos e análise de resultados práticos, de forma que o conhecimento seja construído sequencialmente, como se segue:

- Capítulo 2 - Configurando o Ambiente de Trabalho: Apresenta as instruções necessárias para preparar o ambiente de desenvolvimento, abordando a integração do *Python* dentro do *RStudio* (via pacote *reticulate*) e o gerenciamento de bibliotecas.
- Capítulo 3 - Fundamentos do *Python* para Análise de Dados: Revisa as operações básicas, tipos de dados e a criação de funções, estabelecendo a base sintática da linguagem.
- Capítulo 4 - Manipulação e Análise Exploratória de Dados: Introduz a biblioteca *pandas* e a criação de visualizações, utilizando conjuntos de dados clássicos (como *Titanic* e *Iris*) para demonstrar o tratamento e a exploração de informações categóricas e numéricas.
- Capítulo 5 - Computação Numérica com *NumPy*: Foca no uso de arrays multidimensionais e operações vetorizadas, ilustrando seu poder computacional através de simulações estatísticas.
- Capítulo 6 - Coleta de Dados via *Web Scraping*: Demonstra como construir bases de dados próprias automatizando a extração de informações de páginas da internet, sempre pautado por boas práticas e ética na raspagem de dados.
- Capítulo 7 - Estrutura de Livros Digitais (via *Quarto*): Explica brevemente a estrutura de arquivos e códigos para o desenvolvimento de livros digitais, utilizando o *Rmarkdown* para os códigos e textos e a ferramenta *Quarto* para a compilação.

Espera-se que, ao final deste conteúdo, o estudante ou profissional de Estatística sinta-se confortável para importar, manipular e coletar dados em *Python*, além de estar apto a construir livros digitais que podem ser ferramentas ainda mais robustas que os relatórios padrão, tendo base sólida para estudos posteriores mais aprofundados nestas ferramentas e suas possibilidades.

2 Configurando o Ambiente de Trabalho

Antes de iniciarmos a manipulação de dados e as análises, é fundamental garantir que o ambiente computacional esteja configurado corretamente. Para a maioria dos estudantes e profissionais de Estatística, o *RStudio* é o ambiente de desenvolvimento integrado (IDE) mais familiar. A boa notícia é que o *RStudio*, especialmente quando aliado ao *Quarto*, oferece um suporte excelente para a execução de códigos em *Python*.

2.1 O Ambiente *Python* e o pacote *reticulate*

Em alguns casos, especialmente na primeira execução de um script *Python* dentro do *RStudio*, o ambiente pode não estar perfeitamente configurado. Isso ocorre porque o *RStudio* utiliza um pacote chamado *reticulate* para fazer a ponte e integrar o *Python* ao ecossistema do *R*.

O *reticulate* pode acabar criando, de forma automática, um ambiente virtual *Python* próprio, isolado daquele que você eventualmente já tenha instalado em seu sistema operacional. Por conta desse isolamento, esse novo ambiente nem sempre possui todas as bibliotecas necessárias para a análise de dados já pré-instaladas.

2.2 Gerenciamento de Bibliotecas com o *pip*

Caso apareçam mensagens de erro indicando que bibliotecas fundamentais como *pandas*, *jupyter* ou outras não foram encontradas, elas podem ser facilmente adicionadas utilizando o *pip*. O *pip* é o gerenciador de pacotes padrão do *Python*, responsável por baixar, instalar e atualizar bibliotecas externas.

Para realizar a instalação via **Terminal** (aba **Terminal** no *RStudio*, e não no Console), basta executar os seguintes comandos:

```
python -m pip install pandas numpy seaborn matplotlib
python -m pip install jupyter
```

2.3 Instalação Direta em Blocos de Código

Em documentos interativos (como notebooks *Jupyter* ou arquivos *.qmd* do *Quarto*), também é possível executar comandos de terminal diretamente de dentro das células de código (*code chunks*). Para isso, utiliza-se o caractere de exclamação (!) no início da linha.

Por exemplo, o comando a seguir instala a biblioteca *pandas* diretamente a partir do ambiente de execução do documento, sem a necessidade de abrir o terminal do sistema:

```
!python -m pip install pandas
```

Dica importante: Caso o *RStudio* continue apresentando problemas ou pareça estar utilizando uma instalação do *Python* diferente da esperada, pode ser necessário reiniciar a sessão do *R* (*Session* > *Restart R*) para que o *reticulate* reconheça as novas bibliotecas instaladas. Em geral, após essa configuração inicial e a instalação dos pacotes, o código passará a funcionar normalmente.

3 Fundamentos

Neste capítulo, iremos contextualizar a lógica de algoritmos ao *Python*, explicando brevemente a sua sintaxe essencial. A lógica de programação se mantém a mesma independentemente do ambiente computacional, mas detalhes estruturais como a tipagem de dados, a delimitação de blocos e a indexação variam significativamente de uma linguagem para a outra (Sebesta, 2018).

Enquanto linguagens tipadas estaticamente exigem a declaração prévia do tipo de cada variável (seja ela um número inteiro ou um texto), o *Python* utiliza a tipagem dinâmica, inferindo o tipo do dado no momento de sua execução (Lutz, 2013). Além disso, diferentemente de linguagens que utilizam chaves para agrupar comandos, o *Python* adota a indentação obrigatória como regra sintática, o que impõe a elaboração de códigos padronizados e altamente legíveis. Outro fator de atenção para os analistas e desenvolvedores é a indexação: ao contrário do *R* (que é também uma linguagem amplamente utilizada no contexto de análise de dados e Estatística), onde o primeiro elemento de uma estrutura de dados ocupa o índice 1, o *Python* adota a indexação baseada em zero (0-based), característica compartilhada com linguagens como *C* e o *C++*.

3.1 Filosofia, Sintaxe e Tipagem da Linguagem

Grande parte das linguagens de programação surgem (ou evoluem) focadas em otimização de performance para a máquina. Porém, esse não é o caso do *Python*, que foi concebido a partir de uma filosofia muito bem fundamentada e orientada ao desenvolvedor. O princípio norteador da linguagem é o de que um código é lido muito mais vezes do que ele é escrito. Portanto, a clareza e a legibilidade devem ser prioridades absolutas.

Essa filosofia está muito bem documentada no Zen do *Python* (PEP 20), um conjunto de princípios que guiam as melhores práticas na linguagem, destacando máximas como “o simples é melhor que o complexo” e “a legibilidade conta” (Peters, 2004).

Para garantir essa legibilidade de forma estrutural, a sintaxe do *Python* elimina o uso de marcadores visuais excessivos, como as chaves `{}` ou as expressões `begin/end` para delimitar blocos de código. Em seu lugar, a linguagem adota a indentação obrigatória. O recuo da margem esquerda não é meramente uma questão de estilo, mas uma regra sintática rígida documentada na PEP 8 (Van Rossum; Warsaw; Coghlan, 2001). Seu objetivo é forçar o programador a organizar, hierarquicamente, as operações, resultando em um código mais limpo.

Além da sintaxe visual, o *Python* caracteriza-se por uma tipagem dinâmica e forte. Dinâmica porque não exige a declaração prévia do tipo da variável - a própria linguagem infere se o dado é numérico ou textual no momento da atribuição, além de permitir mudanças de valor e de tipo durante o processo de desenvolvimento. Forte porque, uma vez inferido o tipo, o *Python* não realiza conversões implícitas que possam gerar comportamentos inesperados, garantindo a segurança e a consistência das operações.

3.2 Estruturas de Dados Nativas

Conhecer as estruturas de dados nativas do *Python* é essencial para armazenar e organizar informações. As principais coleções embutidas na linguagem são as listas (`list`), as tuplas (`tuple`), os dicionários (`dict`) e os conjuntos (`set`).

No contexto da manipulação de dados e da programação geral, a **lista** destaca-se como a estrutura mais versátil e frequentemente utilizada. Diferentemente dos vetores atômicos encontrados no *R* ou dos *arrays* estáticos comuns no *C++*, as listas em *Python* são coleções heterogêneas e dinâmicas. Isso significa que uma única lista pode armazenar, de forma simultânea, números inteiros, valores reais, textos ou até mesmo outras listas em seu interior (Lutz, 2013).

Além da heterogeneidade, as listas são estruturas **mutáveis**. Após sua criação, o programador ou analista pode alterar seu conteúdo em tempo de execução sem a necessidade de realocar a estrutura inteira na memória. É possível modificar valores existentes através de seus índices, adicionar novos elementos ao final da coleção (utilizando o método `append()`) ou remover o último item inserido (através do método `pop()`) (Python Software Foundation, 2026a). Essa flexibilidade de manipulação direta torna as listas uma base poderosa para o tratamento inicial de dados antes de avançarmos para o uso de bibliotecas.

3.3 Estruturas de Controle de Fluxo

A espinha dorsal de qualquer linguagem de programação é a capacidade de direcionar a execução dos algoritmos, através de estruturas de controle de fluxo. Destacam-se, nesse contexto, as condições lógicas e os laços de repetição, que são amplamente conhecidos e utilizados por quem trabalha com códigos, seja qual for o contexto. Em *Python*, as estruturas de controle de fluxo seguem a filosofia da linguagem - citada neste capítulo - sendo projetadas para serem diretas e facilmente legíveis, se comparadas a outras linguagens de programação.

As estruturas condicionais utilizam as palavras-chave `if`, `elif` (uma abreviação de `else if`) e `else`. A condição avaliada não exige o uso de parênteses, e o bloco de código subordinado é demarcado de forma simples e intuitiva: através da inserção de dois pontos (`:`) ao final da instrução, seguida da já mencionada indentação obrigatória.

No que diz respeito às estruturas de repetição, o *Python* oferece os laços `while` e `for`. O laço `while` opera de maneira tradicional, executando um bloco de código enquanto uma condição permanecer verdadeira. Contudo, o laço `for` apresenta uma diferença fundamental e vantajosa em relação a linguagens compiladas clássicas. Em vez de iterar primariamente sobre uma progressão aritmética de índices, o `for` em *Python* atua como um iterador direto sobre os itens de qualquer sequência, como uma lista ou uma tupla (Python Software Foundation, 2026a).

No âmbito da análise de dados, por exemplo, essa característica permite que o analista percorra os dados de uma coleção diretamente (por exemplo, lendo cada observação de uma variável) sem a necessidade de gerenciar variáveis contadoras, o que reduz significativamente a chance de erros lógicos. Caso uma iteração numérica tradicional seja estritamente necessária, a linguagem fornece uma função embutida `range()`, capaz de gerar sequências numéricas de forma eficiente. É importante destacar que a função `range()` não armazena a sequência inteira na memória de uma só vez; ela calcula os valores sob demanda a cada passo da iteração. Consequentemente, sua complexidade de

espaço (consumo de memória) é constante, $O(1)$ ¹, independentemente de a sequência possuir dez ou dez milhões de elementos (Python Software Foundation, 2026b). Isso evita gargalos de performance e torna o código otimizado.

3.4 Funções e Ferramentas Funcionais

O processo de desenvolvimento de um algoritmo não apresenta **obrigatoriedade** de práticas que vão além de uma lógica concisa e do respeito à sintaxe da linguagem. Porém, a escalabilidade de um *software* ou a estrutura de uma análise de dados, por exemplo, exigem boas práticas que são fundamentadas no bom uso das funções (métodos), antes de partir para estruturas mais complexas. No *Python*, a definição de uma função é realizada através da palavra-chave **def**, seguida do nome da função e de seus respectivos parâmetros. Assim como nas funções prontas do *Python*, as funções customizadas também exigem, por padrão da sintaxe, os parênteses logo após o nome da função. Portanto, os parâmetros são criados dentro dos parênteses, e o valor de saída da operação, ao final da função, é especificado pelo comando **return**.

Contudo, como citado neste capítulo, o *Python* não se limita ao paradigma procedural; ele incorpora elementos poderosos do paradigma de programação funcional. Para evitar a escrita de laços **for** muito extensos ao transformar bases de dados, a linguagem fornece funções de alta ordem (*higher-order functions*), sendo as mais notáveis o **map()** e o **filter()** (Python Software Foundation, 2026c).

Para aqueles que estão habituados ao uso da linguagem *R*, muito usada na Estatística e na análise de dados, essa mecânica é conceitualmente idêntica ao uso de funções **apply()** (como o **lapply()** e o **sapply()**). O objetivo é o mesmo: permitir um processamento de dados mais rápido, conciso e com menos linhas de código. O **map()** aplica uma mesma transformação a todos os itens de uma coleção de forma simultânea, enquanto o **filter()** extrai e retorna apenas os elementos que satisfazem uma determinada condição lógica (bem próximo do que faz a função **which()**, no *R*).

Essas ferramentas são ainda mais ágeis quando combinadas com as expressões **lambda**. Uma função **lambda** é uma função anônima — ou seja, uma função que não precisa ser formalmente nomeada com **def** e que é criada em uma única linha de código, sendo ideal para operações rápidas ou descartáveis que serão usadas apenas uma vez dentro de um **map** ou **filter** (Lutz, 2013).

3.5 Aplicação Prática: Manipulação de Listas

No script a seguir, consolidamos todos os fundamentos apresentados neste capítulo. O exemplo ilustra desde a manipulação direta de uma lista utilizando métodos nativos (como o **append()** e o **pop()**), até o uso avançado das funções **map()** e **filter()** associadas a funções anônimas (**lambda**). Por fim, demonstramos a aplicação de uma função customizada, desenvolvida com **def()**, para o cálculo de uma média simples.

¹A notação $O(1)$ faz parte da chamada “Notação *Big O*” (*Big O Notation*), utilizada na Ciência da Computação para descrever matematicamente a eficiência e a escalabilidade de um algoritmo. Uma complexidade de espaço $O(1)$, ou constante, indica que a quantidade de memória exigida para a execução da tarefa permanece exatamente a mesma, não importando se a entrada possui dez ou dez milhões de elementos.

Nota-se, também, o uso das chamadas *f-strings* (`f"texto {variável}"`) dentro da função `print()`. Esse é o método mais moderno e recomendado no *Python* para interpretar textos e variáveis na mesma linha, gerando saídas de telas mais limpas.

```
# ---- Definição de uma função customizada ----
def media(lista):
    soma = sum(lista)
    quantidade = len(lista)
    return soma / quantidade

# ---- Criação e manipulação direta da lista ----
valores = [10, 20, 30, 40]
print(f"Lista original: {valores}")

# Uso do append()
valores.append(50)
print(f"Após append (adicionar 50): {valores}")

# Uso do pop()
valores.pop()
print(f"Após pop (remover último elemento): {valores}")

# ---- Uso de ferramentas funcionais (map, filter e lambda) ----

# Uso do map() com lambda
valores_dobrados = list(map(lambda x: x * 2, valores))
print(f"Valores dobrados com map: {valores_dobrados}")

# Uso do filter() com lambda
valores_filtrados = list(filter(lambda x: x > 20, valores))
print(f"Valores > 20 com filter: {valores_filtrados}")

# ---- Aplicação da função customizada ----
resultado = media(valores)
print(f"Média final dos valores originais: {resultado}")
```

```
Lista original: [10, 20, 30, 40]
Após append (adicionar 50): [10, 20, 30, 40, 50]
Após pop (remover último elemento): [10, 20, 30, 40]
Valores dobrados com map: [20, 40, 60, 80]
Valores > 20 com filter: [30, 40]
Média final dos valores originais: 25.0
```

4 Análise Exploratória e Manipulação de Dados em *Python*

Chegamos no ponto-chave de qualquer análise de dados. Antes da modelagem, da inferência e da extração de resultados estatísticos, é a etapa exploratória que contextualiza o analista. Sem a compreensão prévia do cenário, qualquer procedimento será puramente matemático: exato em termos teóricos, porém vazio de informação aplicável ou interpretável. A função da Estatística é justamente atribuir significado aos dados, contando a sua história por meio de metodologias robustas e bem fundamentadas.

Dessa maneira, é preciso compreender a essência das informações em mãos. É através da Análise Exploratória de Dados (AED) que esse processo ganha vida e, no ecossistema *Python*, ele é conduzido primordialmente com o suporte de bibliotecas especializadas na manipulação e visualização de estruturas tabulares[^][Uma estrutura tabular refere-se à organização bidimensional dos dados em formato de tabela. Na terminologia estatística, equivale a uma matriz de dados, onde cada linha representa uma observação (ou registro) e cada coluna representa uma variável (ou atributo) de interesse.

4.1 A Biblioteca *Pandas* e as Estruturas de Dados

Durante muito tempo, a manipulação de dados tabulares não era uma tarefa trivial em *Python*, sendo outras linguagens — como o *R* — preferidas para análises estatísticas. Para suprir a necessidade de ferramentas analíticas flexíveis e de alta performance, o pesquisador Wes McKinney iniciou, em 2008, o desenvolvimento da biblioteca **pandas**. Hoje, ela se consolidou como o padrão ouro da indústria para a limpeza, transformação e análise de dados em *Python* (McKinney, 2022). O ecossistema do **pandas** é construído sobre duas estruturas de dados fundamentais: a **Series** e o **DataFrame**.

Uma **Series** pode ser definida como um vetor unidimensional (semelhante a uma coluna de uma tabela) capaz de armazenar qualquer tipo de dado, acompanhada por um vetor de rótulos chamado de índice (*index*).

Já o **DataFrame** — a estrutura principal da biblioteca — é um objeto bidimensional tabular. Para o estatístico, ele pode ser compreendido como a equivalência exata de um **data.frame** na linguagem *R* ou de uma planilha eletrônica estruturada. Matematicamente, um **DataFrame** é essencialmente um dicionário onde cada coluna é uma **Series** que compartilha o mesmo índice posicional (McKinney, 2022).

Apesar da semelhança teórica com o *R*, a manipulação dessas estruturas apresenta peculiaridades sintáticas importantes. Enquanto no *R* a indexação se inicia em 1, a do **pandas** segue a regra do

Python (baseada em zero), e a seleção explícita de dados é intermediada por métodos específicos da biblioteca, como os localizadores `.loc` (para rótulos) e `.iloc` (para posições inteiras). A partir dessa estrutura bidimensional, o analista ganha acesso imediato a métodos otimizados para a seleção de colunas, filtragem de linhas, criação de novas variáveis e tratamento de valores ausentes.

4.2 Importação e Tratamento de Dados (*Data Wrangling*)

Antes que qualquer inferência estatística ou visualização seja possível, os dados brutos precisam ser carregados para a memória e rigorosamente limpos. Na comunidade analítica, esse processo de estruturação e higienização é frequentemente chamado de *Data Wrangling*.

O primeiro passo é a importação dos dados. O **pandas** oferece diversas opções para a leitura de dados, sendo o método `pd.read_csv()` o mais trivial deles. Ao contrário das funções básicas de leitura em outras linguagens, o **pandas** possui um motor de inferência de tipos altamente otimizado, lidando de forma automática com a alocação de memória para variáveis numéricas, textuais ou séries temporais no momento da leitura (VanderPlas, 2016).

Uma vez importada a matriz de dados, é muito comum nos depararmos com células de valores ausentes (*missing values*). No *R*, estes valores são representados como **NA**, enquanto no *Python* eles são representados como **NaN**. Ambos representam a mesma coisa, mudando apenas a forma de apresentação. É evidente que o tratamento de valores ausentes é um tópico de muita discussão teórica na Estatística, já que alguns métodos, como o de preenchê-los com a média, por exemplo, podem nos levar a subestimar a variância em alguns casos, o que introduziria um viés na análise dos dados. Portanto, este tipo de tratamento deve sempre ser realizado com atenção considerando que ele não depende apenas da aplicação de funções computacionais.

Neste contexto, dado que o analista fez, previamente, uma análise robusta que garanta a viabilidade de algum método de preenchimento de valores ausentes, o **pandas** oferece ótimas ferramentas práticas para a aplicação dos métodos: a função `dropna()` permite a exclusão direta das observações ou variáveis incompletas, enquanto a `fillna()` viabiliza técnicas de imputação, como o preenchimento de falhas utilizando a média, a mediana ou o último valor válido observado da respectiva série.

Outro pilar crítico do *Data Wrangling* é o cruzamento de bases relacionais. Projetos reais raramente dependem de um único arquivo isolado; é frequente a necessidade de unir tabelas distintas por meio de variáveis-chave, utilizando a função `pd.merge()`. Nesta fase, a integridade lógica da base final deve ser a prioridade. Como regra fundamental de validação de dados em junções, caso ocorram discrepâncias ou identificadores duplicados ao combinar duas bases, os valores oriundos da primeira tabela (considerada a base primária ou de maior confiabilidade) devem sempre prevalecer. No *Python*, esse controle lógico rigoroso é facilitado pela arquitetura de *joins* do **pandas** e por métodos de alinhamento de índices, garantindo que o tratamento de duplicidades ou conflitos não comprometa a informação original mais segura.

4.3 Visualização dos Dados

A qualidade de visualização dos dados é essencial na Estatística. É através dos recursos gráficos que podemos identificar tendências, anomalias e até mesmo distribuições. A visualização constitui a fase inicial de qualquer análise descritiva dos dados, mas não se limita a apenas isso. É comum,

por exemplo, utilizar o QQ-plot para avaliar a normalidade dos resíduos de um modelo, de forma a dar suporte aos testes não-paramétricos que podem levar o analista à conclusões equivocadas em alguns casos, se utilizados de forma isolada. É justamente por este motivo que a utilização de mais de um recurso é sempre recomendada para testar pressuposições teóricas importantes.

No âmbito visual, o *Python* conta com bibliotecas muito poderosas. São elas, principalmente, o **matplotlib** e o **Seaborn**. O **matplotlib** atua como o motor estrutural da linguagem. Ele é responsável por instanciar e desenhar os elementos fundamentais da interface gráfica, como a figura (a “tela” em branco), os eixos cartesianos, as marcações e as escalas (Hunter, 2007). Por ser uma ferramenta de baixo nível (termo que, na computação, refere-se à menor distância conceitual entre o código e a máquina, indicando a necessidade de especificar instruções mais minuciosas, e não uma avaliação sobre a qualidade da ferramenta) e extremamente detalhista, ele exige uma quantidade considerável de código para gerar representações esteticamente polidas, assemelhando-se aos pacotes base de gráficos do R (como a função `plot()`).

Para suprir a necessidade de gráficos estatísticos mais expressivos e com menos linhas de código, surgiu o **seaborn**. Construída nativamente sobre o **matplotlib**, essa biblioteca funciona como uma interface declarativa de alto nível, sendo perfeitamente integrada aos objetos da biblioteca **pandas** (Waskom, 2021).

O grande diferencial do **seaborn** para o analista de dados é a abstração de complexidades estruturais somada à automatização de agregações estatísticas. Em muitas de suas funções, a biblioteca calcula e exibe intervalos de confiança de forma automática ao agregar dados. Além disso, o **seaborn** traz paletas de cores nativas fundamentadas na teoria da percepção visual, garantindo que as distinções categóricas e as intensidades contínuas sejam compreendidas de maneira intuitiva e sem vieses de leitura.

4.4 Estudos de Caso Práticos

Para consolidar os conceitos de *Data Wrangling* e visualização, aplicaremos as ferramentas do **pandas** e do **seaborn** em dois conjuntos de dados clássicos na literatura estatística e no aprendizado de máquina: a base **Titanic** (focada em dados categóricos e taxas de sobrevivência) e a base **Iris** (focada em dados contínuos e distribuições dimensionais).

Para facilitar a reprodutibilidade dos códigos a seguir, utilizaremos as versões dessas bases que já vêm nativamente embutidas na biblioteca **seaborn**.

4.4.1 Análise de Dados Categóricos: *Titanic*

O conjunto de dados do naufrágio do **Titanic** é ideal para a exploração de tabelas de contingência e distribuições de frequência. No *Python*, a função `pd.crosstab()` do **pandas** atua de forma muito semelhante à clássica função `table()` do R, permitindo o cruzamento rápido entre variáveis qualitativas.

No script abaixo, importamos as bibliotecas necessárias, carregamos os dados, removemos rapidamente os valores ausentes de uma coluna de interesse estrutural e geramos a tabela de contingência cruzando a classe do passageiro com o status de sobrevivência. Em seguida, utilizamos o **seaborn** para criar um gráfico de barras.

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Carregando a base de dados
titanic = sns.load_dataset('titanic')

# Tratamento rápido (Data Wrangling): Removendo linhas sem informação de embarque
titanic_limpo = titanic.dropna(subset=['embark_town'])

# Tabela de contingência (Classe vs Sobrevivência)
tabela_sobrevivencia = pd.crosstab(
    titanic_limpo['class'],
    titanic_limpo['survived'],
    rownames=['Classe'],
    colnames=['Sobreviveu?']
)
print("Tabela de Contingência - Titanic:\n")
print(tabela_sobrevivencia)

# Configurando e gerando o gráfico com Seaborn
plt.figure(figsize=(8, 5))
sns.countplot(
    data=titanic_limpo,
    x='class',
    hue='survived',
    palette='Set2'
)
plt.title("Sobrevivência por Classe de Passageiro")
plt.xlabel("Classe")
plt.ylabel("Contagem")
plt.show()

```

Tabela de Contingência - Titanic:

Sobreviveu?	0	1
Classe		
First	80	134
Second	97	87
Third	372	119



4.4.2 Análise de Dados Contínuos: *Iris*

A base *Iris*, introduzida pelo estatístico *Ronald Fisher* em 1936, é o exemplo definitivo para o estudo de distribuições contínuas e análise de correlação entre múltiplas variáveis numéricas (como comprimento e largura de pétalas e sépalas).

Antes de partirmos para a visualização, a extração de métricas de posição e dispersão é o passo primário para a compreensão de variáveis contínuas. No *R*, essa tarefa é comumente realizada através da função `summary()`. No *Python*, a biblioteca `pandas` fornece o método `.describe()`, que calcula instantaneamente a contagem, a média, o desvio padrão e os quartis de todas as colunas numéricas do *DataFrame*.

Enquanto as métricas descritivas resumem os dados, a exploração de dados contínuos exige uma abordagem visual para a identificação de padrões lineares ou de agrupamentos (*clusters*). O gráfico de dispersão (*scatterplot*) é a ferramenta analítica primária para esse fim. O `seaborn` permite estratificar esses pontos facilmente através do parâmetro `hue`, colorindo as observações com base em uma variável categórica, como a espécie da flor.

```
# Carregando a base de dados
iris = sns.load_dataset('iris')

# Extração das métricas descritivas (equivalente ao summary do R)
print("Resumo Estatístico da Base Iris:\n")
```

```

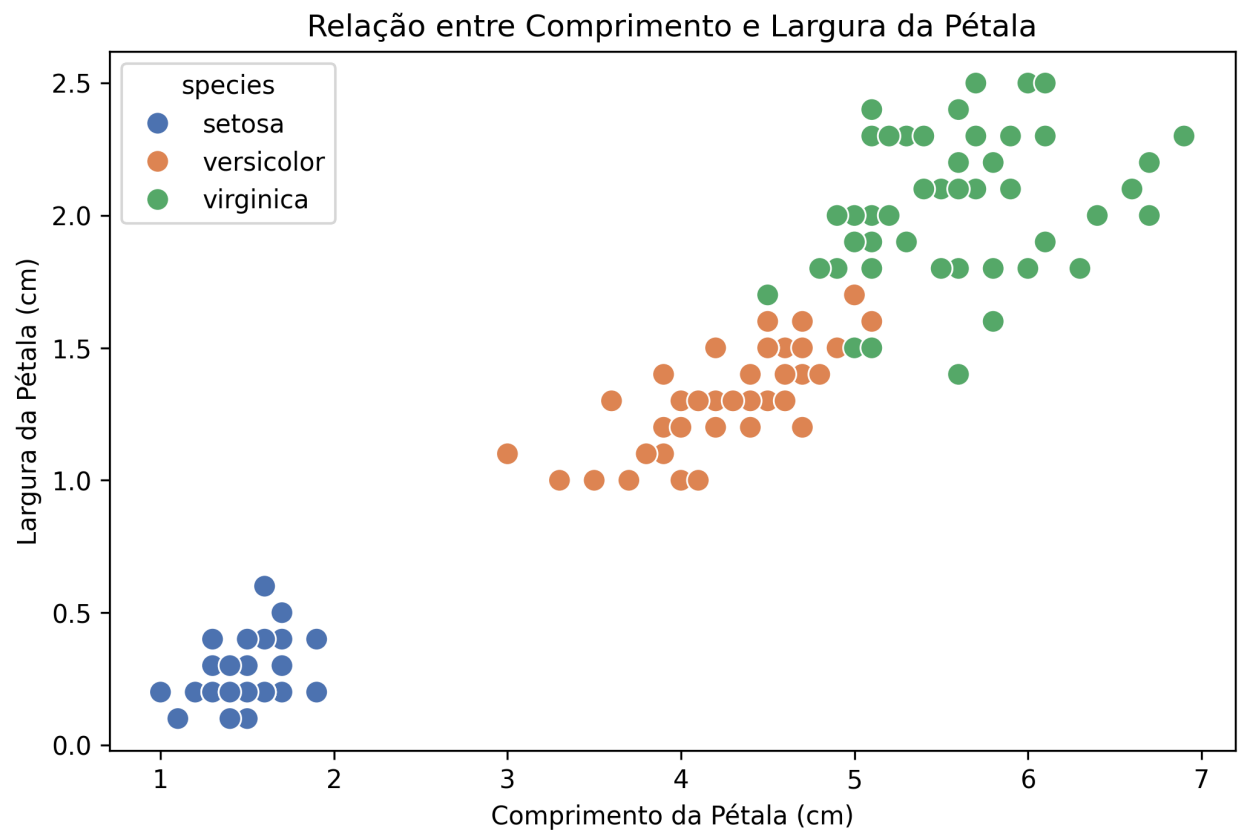
print(iris.describe())

# Configurando e gerando o gráfico de dispersão com Seaborn
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=iris,
    x='petal_length',
    y='petal_width',
    hue='species',
    palette='deep',
    s=80 # Tamanho dos pontos
)
plt.title("Relação entre Comprimento e Largura da Pétala")
plt.xlabel("Comprimento da Pétala (cm)")
plt.ylabel("Largura da Pétala (cm)")
plt.show()

```

Resumo Estatístico da Base Iris:

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000



5 Referências

HUNTER, John D. Matplotlib: A 2D graphics environment. **Computing in science & engineering**, v. 9, n. 3, p. 90–95, 2007.

LUTZ, Mark. **Learning Python: Powerful Object-Oriented Programming**. 5. ed. *[S.l.]*: O'Reilly Media, Inc., 2013.

MCKINNEY, Wes. **Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter**. 3. ed. *[S.l.]*: O'Reilly Media, Inc., 2022.

PETERS, Tim. **PEP 20: The Zen of Python**. *[S.l.]*: Python Software Foundation, 2004. Disponível em: <<https://peps.python.org/pep-0020/>>.

PYTHON SOFTWARE FOUNDATION. **The Python Tutorial: Data Structures**. *[S.l.: S.n.]*.

PYTHON SOFTWARE FOUNDATION. **Built-in Types - Ranges**. *[S.l.: S.n.]*.

PYTHON SOFTWARE FOUNDATION. **Built-in Functions**. *[S.l.: S.n.]*.

SEBESTA, Robert W. **Conceitos de Linguagens de Programação**. 11. ed. *[S.l.]*: Bookman Editora, 2018.

VAN ROSSUM, Guido; WARSAW, Barry; COGHLAN, Nick. **PEP 8: Style Guide for Python Code**. *[S.l.]*: Python Software Foundation, 2001. Disponível em: <<https://peps.python.org/pep-0008/>>.

VANDERPLAS, Jake. **Python Data Science Handbook: Essential Tools for Working with Data**. *[S.l.]*: O'Reilly Media, Inc., 2016.

WASKOM, Michael L. seaborn: statistical data visualization. **Journal of Open Source Software**, v. 6, n. 60, p. 3021, 2021.